

RESULTS

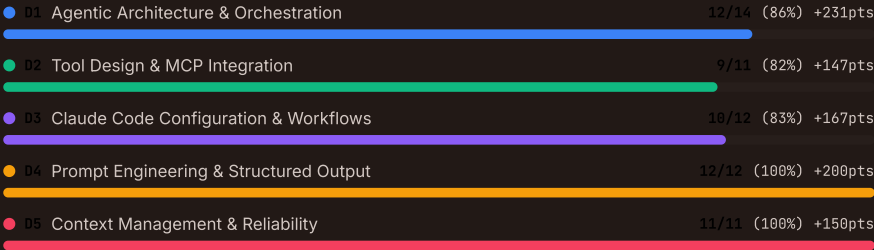
Exam Results

895 / 1000

PASSED

54 of 60 correct (90%) · Pass mark: 720

Domain Breakdown



HOW SCORING WORKS

Each domain's percentage correct is multiplied by its weight and scaled to 1000 points. For example, scoring 80% in Agentic Architecture (27% weight) contributes $0.80 \times 0.27 \times 1000 = 216$ points. The pass mark is 720/1000 (72%). If a domain has no questions in the selected scenarios, its weight is redistributed proportionally among assessed domains.

The real exam reports a scaled score from 100 to 1,000 plus your percent-correct by domain, so treat the percentages here as a readiness gauge rather than a prediction of your scaled score.

HIDE DETAIL

TAKE ANOTHER EXAM

Question-by-Question Review

ALL 60

CORRECT 54

INCORRECT 6

SKIPPED 0

Q01 · D1 1.7 SESSION-STATE-RESUMPTION / FRESH-CONTEXT q-1-7-008 · Content Moderation and Classification System CORRECT

The moderation system handles appeals where users contest a moderation decision. Currently, the same agent that made the original decision re-evaluates the appeal. Appeal overturn rates are suspiciously low. What is the most effective architectural change?

C Route appeals to a separate agent instance that cannot see the original reasoning, given only the content and the user's justification.

A fresh agent instance without access to the original reasoning context evaluates the content independently. This avoids confirmation bias from the original decision. The appeal agent sees only the content and the user's argument, enabling a genuinely independent review.

B Route appeals to a human reviewer who has full authority to overturn automated decisions

Human review for every appeal does not scale and removes the benefit of automated moderation. Some appeals can be resolved automatically by a separate, unbiased instance.

A Add stronger instructions to the appeal handler's prompt requiring it to weigh the user's perspective fairly and set aside its earlier decision.

Prompt instructions cannot overcome the bias of an agent reviewing its own decision. The original reasoning context still influences the re-evaluation regardless of instructions.

D Automatically overturn decisions where the user provides any appeal justification to improve user trust

Automatic overturn on any appeal completely undermines moderation. Users who genuinely violated policies would simply appeal every decision.

WHERE THIS COMES FROM

Lesson 1.7: Session State and Resumption (Stale context)

Lesson 1.2: Multi-Agent Orchestration (Isolation principle)

Q02 · D2 2.2 STRUCTURED-ERROR-RESPONSES / TRANSIENT-ERROR q-2-2-005 · Multi-Agent Research System CORRECT

A research agent calls an external API via an MCP server. After the 30th query in a batch of 50, the API starts returning HTTP 429 errors. The MCP server returns a generic 'Request failed' for every failure, so the agent abandons the batch after three consecutive failures. What MCP server change would most improve resilience?

A Implement automatic retry with exponential backoff inside the MCP server, hiding rate limits from the agent entirely.

Hiding rate limits from the agent removes its ability to make informed decisions, such as prioritising remaining queries, switching to cached results, or reporting partial progress. The agent should receive structured metadata to decide its own recovery strategy.

D Queue all 50 requests at the MCP server level and process them sequentially with built-in rate limiting.

Server-side queuing removes the agent's ability to prioritise, cancel, or reorder queries based on intermediate results. It also creates a long-running blocking call that prevents the agent from doing other work.

B Return errorCategory: 'transient', isRetryable: true, with a retryAfterMs field telling the agent how long to wait.

Rate limiting is a transient error — it will resolve after a delay. Structured metadata with the specific retry delay lets the agent space out remaining queries intelligently rather than abandoning the batch. The agent can continue processing other tasks while waiting.

C Return errorCategory: 'business', isRetryable: false to tell the agent to stop making requests entirely.

Rate limits are transient (they resolve after a delay), not business errors (which represent policy or rule violations the agent cannot recover from). Marking them as non-retryable causes the agent to abandon 20 remaining queries that would succeed after a brief delay.

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Transient errors and retry-after)

Q03 · D4 4.6 MULTI-PASS-REVIEW / SELF-REVIEW-BIAS q-4-6-005 · Content Moderation and Classification System CORRECT

The moderation team asks a single Claude session to classify a post, then immediately asks the same session to independently review its own classification for quality assurance. The 'review' agrees with the original classification 98% of the time, including cases that human auditors later identify as errors. Why is this self-review ineffective?

D Self-review is effective but the 98% agreement rate simply reflects high initial accuracy — the 2% disagreement is the expected error rate

Human auditors found errors in cases where the self-review agreed, proving the 98% agreement does not reflect accuracy. The self-review is confirming errors, not catching them, due to same-session reasoning bias.

C The model's temperature is too low, producing deterministic agreement — increase temperature for the review pass

Temperature affects sampling randomness, not reasoning independence. Higher temperature may occasionally produce different outputs but does not create genuine independent review. The model is still anchored to its own prior reasoning in the same session.

A The model needs a stronger review prompt with explicit instructions to look for errors in the original classification

Stronger instructions do not overcome the fundamental limitation. The same session retains the reasoning context from the original classification, biasing the review toward agreement.

B The same session retains the model's reasoning, so it stays anchored to its classification; use a fresh independent instance to review.

Self-review within the same session is fundamentally limited because the model's conversation context includes the original reasoning. It will naturally be anchored to its prior conclusions. An independent instance with no access to the original decision evaluates the content fresh, providing genuine quality assurance.

WHERE THIS COMES FROM

Lesson 4.6: Multi-Instance Review and Output Validation (Self-review limitation)

Q04 · D5 5.6 INFORMATION-PROVENANCE / CONFLICT-HANDLING q-5-6-001 · Multi-Agent Research System CORRECT

A multi-agent research system produces a synthesis report on market trends. Two credible sources report different growth rates for the same sector: Source A reports 12% growth (2023 data) and Source B reports 8% growth (2024 data). The synthesis agent currently selects the more recent value. What is the correct approach?

D Flag the conflict and escalate to a human researcher for resolution before including in the report

This is unnecessary — the values are not contradictory, they reflect different time periods. Proper annotation resolves the ambiguity.

C Annotate both values with source and publication dates, letting the consumer interpret them.

This preserves provenance, temporal context, and both data points. The consumer can see that different dates explain different numbers.

A Always use the most recent source as it reflects the latest data

This silently discards valid historical context. The difference may reflect an actual trend, not a contradiction.

B Average the two values and report 10% growth with a note about source variance

Averaging conflicting statistics is mathematically misleading and destroys the temporal context that explains the difference.

WHERE THIS COMES FROM

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Conflict handling)

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Temporal awareness)

Q05 · D2 2.1 TOOL-SCHEMA-DESIGN / DESCRIPTIONS q-2-1-005 CORRECT

A customer support agent has two tools: `get_customer_profile` (returns name, email, preferences) and `get_customer_orders` (returns order history with statuses). Users frequently ask 'Tell me about customer X', and the agent calls both tools sequentially, doubling response time. 90% of 'tell me about' queries only need the profile. What is the best approach?

D Set `tool_choice` to forced selection of `get_customer_profile` for all turns to prevent unnecessary order lookups.

Forced selection prevents the agent from ever calling `get_customer_orders`, even when users specifically ask about orders. This solves the over-calling problem by crippling legitimate functionality.

C Add a system prompt instruction: 'When a user says tell me about, only call `get_customer_profile`.'

Hard-coding keyword-to-tool mappings in the system prompt is brittle. It does not generalise to other phrasings like 'What do you know about customer X?' and will break if tool names change.

B Refine the descriptions so `get_customer_profile` is the default and `get_customer_orders` is order-specific.

Refining tool descriptions with clear defaults is the lowest-effort fix. `get_customer_profile` should be marked the default for general 'tell me about' queries, directing the agent to call only the profile tool unless order details are explicitly requested.

A Consolidate both tools into a single `get_customer_info` tool that returns all customer data in one call.

Consolidation returns order data on every call, wasting resources for the 90% of queries that only need profile data. It also increases the response payload size unnecessarily.

WHERE THIS COMES FROM

Lesson 2.1: Tool Interface Design (Tool descriptions)

Q06 · D3 3.1 CLAUDE-MD-HIERARCHY / MEMORY-COMMAND q-3-1-020 · Technical Documentation Maintenance System

CORRECT

A developer's Claude Code applies the team's API conventions correctly in some sessions but not others, on the same project. They suspect the wrong memory files are loading in the failing sessions. What is the fastest way to confirm which CLAUDE.md and rules files a session has actually loaded?

B Run `/compact` to reload the configuration hierarchy from disk

`/compact` summarises the conversation to free context; it does not reload or report configuration files. CLAUDE.md content survives compaction, so this neither diagnoses nor fixes a loading difference.

C Delete `~/.claude/CLAUDE.md` so only project-level configuration can load

Deleting user-level configuration is a destructive guess. It might mask the symptom, but it destroys the developer's personal setup without ever confirming what the failing sessions were loading.

A Run `/memory` in the session to list the loaded memory files

`/memory` is the diagnostic command for the configuration hierarchy: it shows which user-level, project-level, and directory-level memory files the current session has loaded, which directly confirms or rules out the suspected loading difference.

D Ask Claude in the session to repeat the team's API conventions back

The model can paraphrase conventions from training or partial context, so a fluent answer does not prove the files loaded. `/memory` reports the loaded files deterministically instead of relying on the model's self-report.

WHERE THIS COMES FROM

Lesson 3.1: CLAUDE.md Hierarchy and Scoping (`/memory` and CLAUDE.md persistence)

Claude Code: Memory and CLAUDE.md ↗

Q07 · D3 3.2 SLASH-COMMANDS-SKILLS / CONTEXT-FORK q-3-2-015 · Technical Documentation Maintenance System

CORRECT

A documentation team creates a `/generate-api-docs` skill that reads source code files and produces Markdown API reference pages. The skill generates verbose output (200+ lines per endpoint) and should be available to every team member who clones the repository. How should the skill be configured?

B Create a SKILL.md in `.claude/skills/` with context: fork, so it is shared via git and isolates the verbose output

`.claude/skills/` is project-scoped and version-controlled, making it available to every developer who clones the repository. context: fork isolates the verbose 200+ line output per endpoint into a forked context, preventing it from consuming the main conversation's context budget.

A Create a SKILL.md in `~/.claude/skills/` with context: fork frontmatter, and instruct each team member to copy it locally

`~/.claude/skills/` is user-scoped and not version-controlled. Requiring manual copying defeats the purpose of team-shared configuration and creates maintenance drift.

D Create a SKILL.md in `.claude/skills/` without any frontmatter, relying on the team to manually manage context overflow

Without context: fork, the verbose output (200+ lines per endpoint across many endpoints) would flood the main conversation context. The context: fork frontmatter is essential for isolating bulky output.

C Add the documentation generation instructions to the root CLAUDE.md so they load automatically in every session

CLAUDE.md is for always-loaded standards, not for on-demand command definitions. Loading verbose generation instructions in every session wastes context tokens when the team is doing non-documentation work.

WHERE THIS COMES FROM

Lesson 3.2: Custom Slash Commands and Skills (context: fork)

Claude Code: Skills ↗

Q08 · D3 3.1 CLAUDE-MD-HIERARCHY / POSTTOOLUSE-ENFORCEMENT q-3-1-014 · CI/CD Pipeline Integration

CORRECT

A fintech company requires that all API response payloads are normalised to snake_case before being logged, and that any file write to the `src/api/` directory is automatically linted. They want these enforcements

to be deterministic and not rely on the model remembering instructions. Which hook configuration achieves both requirements?

C Add both requirements to CLAUDE.md with clear examples, and add a PreToolUse hook that reminds the model about these rules before each tool call

CLAUDE.md instructions and PreToolUse reminders both rely on the model's compliance, which is not deterministic. The requirement explicitly states that enforcement must not depend on the model remembering instructions.

B A PostToolUse hook on file writes to src/api/ that runs the linter on the created file, and a PostToolUse hook on data processing that normalises response payloads to snake_case

Both are PostToolUse hooks because both act on completed outputs. The linter runs after a file write to src/api/ is complete, validating the written content. The normalisation hook processes API response data after it is retrieved, converting fields to snake_case before logging. Both provide deterministic enforcement independent of model instructions.

A A PreToolUse hook on file writes that runs the linter before the write completes, and a PostToolUse hook on API response handling that normalises field names to snake_case

PreToolUse fires before the tool runs, so there is no file content to lint yet. Linting must happen after the file is written (PostToolUse). The ordering of hooks to actions is incorrect here.

D A PreToolUse hook that blocks any file write to src/api/ unless the file has already been linted, and a PreToolUse hook that blocks API calls unless snake_case normalisation is configured

PreToolUse blocking assumes the file exists before the write, which is contradictory. You cannot lint a file that has not been written yet. PostToolUse hooks that process output after completion are the correct approach for validation and normalisation.

WHERE THIS COMES FROM

Claude Code: Hooks

Q09 · D5 5.4 CODEBASE-EXPLORATION / SCRATCHPAD q-5-4-009 · Technical Documentation Maintenance System CORRECT

A docs team asks Claude Code to audit documentation coverage across a 500,000-line codebase. After reading 30 files via the Read tool, the conversation is approaching the context limit and earlier tool results will be summarised away by auto-compaction. The agent still needs that earlier information to complete the audit. What is the best strategy?

B Have the agent write structured findings to a scratchpad after each file, then read it for the final audit.

A scratchpad file persists on disk and can be re-read at any time, unlike in-context tool results that are trimmed as new results arrive. By extracting and recording the essential findings (not the full file contents) after each file read, the agent decouples its knowledge from context window limits. The final audit reads the compact scratchpad rather than needing all source files in context.

D Run /compact after every 10 files to free up context space for the next batch of file reads

/compact summarises the conversation, which may discard the specific findings from earlier file reads. The problem is preserving information, not just freeing space. Compacting without first persisting findings to disk loses the very data the audit needs.

C Process all files in a single Read tool call by passing a glob pattern, so the results arrive in one untrimmed block

The Read tool reads individual files, not glob patterns. Even if multiple files could be read at once, a single massive result block would itself be trimmed or would consume the entire context budget, preventing the agent from reasoning about the results.

A Increase the context window size so that all 30+ file contents can be held simultaneously without trimming

The context window has a fixed maximum size. A 500,000-line codebase cannot fit entirely in context regardless of configuration. Tool result trimming exists specifically because holding all file contents simultaneously is not feasible.

WHERE THIS COMES FROM

Lesson 5.4: Codebase Exploration and Context Degradation (Scratchpad files)

Lesson 5.4: Codebase Exploration and Context Degradation (/compact behaviour)

Q10 · D4 4.5 BATCH-PROCESSING / BATCH-FAILURE q-4-5-002 · CI/CD Pipeline Integration

CORRECT

Your CI/CD pipeline generates nightly security audit reports using the Message Batches API. A batch of 200 documents completes overnight, but 15 documents fail due to exceeding the context window. The team proposes resubmitting the entire batch of 200 documents. What is the correct failure handling strategy?

D Increase the batch processing timeout to give the system more time to handle large documents

The failure is due to documents exceeding the context window, not a timeout issue. The documents need to be chunked to fit within the model's context limits.

B Resubmit only the 15 failed documents identified by custom_id, chunking the oversized documents before resubmission

Identify failed documents by custom_id, then resubmit only failures with modifications (chunking oversized documents). This is the correct batch failure handling pattern — it avoids reprocessing the 185 successful documents and addresses the root cause of the failures.

A Resubmit the entire batch since you cannot identify which documents failed

The batch API uses custom_id fields to correlate request/response pairs. Failed documents can be identified by their custom_id, making full resubmission unnecessary and wasteful.

C Switch the entire pipeline to synchronous processing to avoid batch failures

Nightly security audits are latency-tolerant workloads perfectly suited for batch processing. Switching to synchronous forfeits the 50% cost savings without solving the oversized document problem.

WHERE THIS COMES FROM

Lesson 4.5: Batch Processing and Prompt Optimisation (Batch failure handling)

Lesson 4.5: Batch Processing and Prompt Optimisation (custom_id matching)

Q11 · D2 2.4 MCP-SERVER-INTEGRATION / ENV-VARS q-2-4-005

CORRECT

A platform engineering team wants to share an MCP server for their internal ticketing system across all developers using Claude Code. The server requires an API token unique to each developer. Where should the MCP server be configured, and how should credentials be managed?

B Add the server to .mcp.json using \${TICKETING_API_TOKEN} expansion so each developer sets their own token.

Project-level .mcp.json is version-controlled and shared, making the server available to everyone. Environment variable expansion keeps individual credentials out of version control while letting each developer authenticate with their own token.

A Add the server to .mcp.json in the project root with each developer's API token hard-coded in the configuration.

Hard-coding individual tokens in .mcp.json commits credentials to version control, which is a security risk. Each developer has a different token, so a single hard-coded value would not work for the team.

C Have each developer add the server to their personal ~/.claude.json with their API token.

User-level ~/.claude.json is for personal or experimental servers. A team-wide integration should be in project-level .mcp.json so it is version-controlled and consistently available to all developers.

D Create a shared .env file in the repository with all developer tokens and reference it from .mcp.json.

A shared .env file with all developer tokens still commits credentials to version control and creates a single file containing every team member's secrets, compounding the security risk.

WHERE THIS COMES FROM

Lesson 2.4: MCP Server Integration (Environment variable expansion)

Lesson 2.4: MCP Server Integration (Scoping hierarchy)

Claude Code: MCP Server Configuration ↗

Q12 · D5 5.6 INFORMATION-PROVENANCE / CONFLICT-HANDLING q-5-6-005 · Multi-Agent Research System

CORRECT

A synthesis agent receives findings from three subagents: solar investment grew 15% (Bloomberg, January 2024), solar investment grew 22% (IEA, March 2024), and cost-per-watt data. The synthesis agent currently picks the IEA figure because it is more recent. What is the correct approach?

A Always use the most authoritative source (IEA as an international agency) and discard the Bloomberg figure.

Discarding a credible source based on perceived authority silently removes valid data. Both sources are credible, and the difference may reflect different methodologies, scopes, or time periods.

B Average the two values (18.5%) and cite both sources to present a balanced view.

Averaging conflicting statistics is mathematically misleading. The two figures may measure different scopes (e.g., different regions or investment types), making an average meaningless. The difference needs to be preserved, not merged.

C Present both figures with source attribution and dates, noting what may explain the difference.

Preserving both data points with provenance — source, date, and potential explanations for the difference — lets the consumer interpret the data correctly. Different publication dates and methodologies explain the discrepancy without requiring the synthesis agent to judge which is 'right'.

D Flag the conflicting figures and pause report generation until a human researcher resolves the discrepancy.

Pausing for human resolution is unnecessary when the difference can be explained by temporal or methodological factors. Proper annotation with provenance resolves the ambiguity without blocking the workflow.

WHERE THIS COMES FROM

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Conflict handling)

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Temporal awareness)

Q13 · D4 4.1 SYSTEM-PROMPTS / KEYWORD-OVERLAP q-4-1-004 · CI/CD Pipeline Integration

CORRECT

Your CI/CD pipeline uses a system prompt with a tool named 'check_security' and also includes instructions that say 'check the security of each function'. Developers report that the model sometimes produces a text-based security analysis instead of calling the check_security tool. What is the most likely cause?

D The system prompt has grown too long, so the model skips over the tool definitions near the end and never registers that check_security is available to call.

System prompt length does not cause tool definitions to be skipped. The model processes tools and instructions together. The issue is keyword overlap creating ambiguity.

C The model cannot call tools from within a CI/CD pipeline context

Tool calling works in any context. The issue is prompt design, not an environmental limitation.

A The model's temperature is too high, causing random tool selection

Temperature affects randomness in generation but is not the primary cause of tool call confusion. The issue is prompt design, not sampling parameters.

B The 'check the security' instruction keyword-overlaps the 'check_security' tool name, so the model follows the text instead of calling the tool.

When system prompt instructions use phrasing that closely mirrors tool names, the model can interpret 'check the security' as a general instruction rather than a trigger for the check_security tool. Distinct naming reduces this ambiguity.

WHERE THIS COMES FROM

Lesson 4.1: System Prompts with Explicit Criteria (Keyword overlap)

Anthropic: Tool Use Documentation ↗

Q14 · D3 3.1 CLAUDE-MD-HIERARCHY / POSTTOOLUSE-VALIDATION q-3-1-012 · CI/CD Pipeline Integration

CORRECT

A team wants to enforce that all SQL migration files created by Claude Code follow a strict naming convention (YYYYMMDD_HHMMSS_description.sql) and contain a rollback section. Which approach provides deterministic enforcement without relying on the model's judgment?

C Create a .claude/rules/migrations.md file with paths: ["**/migrations/**/*.sql"] containing the naming convention

Rules files provide context-specific instructions but still rely on the model to follow them. They do not provide deterministic enforcement. A hook-based approach guarantees compliance through automated validation.

A Add a PostToolUse hook on file creation that validates the filename pattern and checks for a rollback section

PostToolUse hooks run deterministic validation code after tool execution. A hook that checks filename patterns and content requirements provides reliable, automated enforcement that does not depend on the model remembering or correctly interpreting naming conventions.

B Document the naming convention and rollback requirement in the project CLAUDE.md with clear examples

CLAUDE.md instructions rely on the model's interpretation and compliance. While useful as guidance, they do not provide deterministic enforcement. The model may occasionally deviate from the convention, especially under complex prompts.

D Use a PreToolUse hook to inject the naming convention into every prompt before file creation occurs

PreToolUse hooks fire before tool execution and are better suited for blocking or modifying tool calls. Injecting instructions into prompts still relies on the model's compliance. PostToolUse validation provides deterministic enforcement by checking the actual output.

WHERE THIS COMES FROM

Claude Code: Hooks ↗

Q15 · D1 1.2 ORCHESTRATION-PATTERNS / COORDINATOR q-1-2-002 · Multi-Agent Research System **CORRECT**

A consulting firm's multi-agent research system has a coordinator that always invokes the full pipeline of five subagents (web search, document analysis, data extraction, synthesis, and formatting) for every query, including simple factual lookups that only need web search. This adds unnecessary latency and cost. What is the correct architectural fix?

B Allow subagents to communicate directly with each other so they can skip unnecessary steps in the pipeline

Direct subagent communication violates the hub-and-spoke architecture. All communication must flow through the coordinator. The fix is smarter coordinator routing, not bypassing the coordinator.

C Add a caching layer so subagents that have no work to do return immediately

Caching does not address the architectural issue. Invoking subagents that are not needed adds API call overhead and latency regardless of whether results are cached. The coordinator should not invoke unnecessary subagents in the first place.

A Have the coordinator select which subagents to invoke per query, not always the full pipeline.

The coordinator should analyse query requirements and decide which subagents are needed. A simple factual lookup only needs the web search agent, not the full five-agent pipeline. Dynamic selection reduces latency and cost for straightforward requests.

D Split into two separate systems: one for simple queries and one for complex queries

Maintaining two separate systems creates unnecessary complexity. The coordinator's role is precisely to analyse query requirements and make routing decisions. A single coordinator with dynamic selection handles both cases.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Q16 · D5 5.5 HUMAN-REVIEW-CALIBRATION / REVIEWER-CAPACITY q-5-5-007 · Technical Documentation Maintenance System **CORRECT**

Claude Code generates API reference pages for 150 endpoints across three categories: payment processing (30 endpoints, high regulatory risk), internal tooling (80 endpoints, low risk), and public data queries (40 endpoints, moderate risk). The team cannot manually review all 150. How should they structure human review?

B Use stratified sampling: review 100% of payment docs, 10% of public data query docs, and 5% of internal tooling docs.

Stratified sampling allocates review effort proportional to risk. Payment processing documentation has regulatory implications warranting full review. Public data queries affect external users and deserve moderate scrutiny. Internal

tooling has the lowest blast radius and can tolerate the lightest review. This achieves thorough coverage of critical content within a manageable review budget.

A Randomly sample 15% of all endpoints (approximately 23 pages) for review, ensuring a representative cross-section

Uniform random sampling treats all endpoints equally, but payment processing endpoints carry much higher risk. A 15% sample might only include 4-5 payment endpoints, inadequately covering the highest-risk category.

D Have Claude Code self-assess each generated page with a confidence score and only review pages where confidence falls below 80%

LLM self-reported confidence is poorly calibrated. Claude Code may report high confidence on pages with subtle factual errors (e.g., incorrect parameter types, wrong default values) because the generated text is fluent and internally consistent.

C Review only the payment processing endpoints since they are the highest risk, and trust Claude Code's output for the remaining 120 endpoints

Completely skipping review for 120 endpoints risks undetected errors in public-facing documentation. Even low-risk internal tooling docs should have some sample review to catch systematic generation issues that might affect all categories.

WHERE THIS COMES FROM

Lesson 5.5: Human Review and Confidence Calibration (Reviewer capacity)

Lesson 5.5: Human Review and Confidence Calibration (Stratified sampling)

Q17 · D1 1.5 AGENT-SDK-HOOKS / PRETOOLUSE-ENFORCEMENT q-1-5-010 · Content Moderation and Classification System

CORRECT

Platform policy strictly prohibits the moderation system from auto-removing any post in the 'political speech' category without human approval. Currently this rule is enforced via the system prompt: 'Never auto-remove political speech without human approval.' Audit logs reveal that in 2% of cases the agent auto-removes political speech posts. What is the correct fix?

A Strengthen the system prompt by adding the rule in multiple locations with bold formatting and emphasis

Repeating the rule more emphatically is still a probabilistic approach. The existing 2% failure rate demonstrates that prompt-based enforcement cannot guarantee 100% compliance for a strict policy requirement.

D Route all political speech posts to a dedicated subagent that has no access to the auto-remove tool

Routing to a restricted subagent is partially correct but relies on the coordinator to correctly identify political speech before routing. A hook on the removal tool itself is more robust because it catches violations regardless of routing decisions.

B Add few-shot examples showing the correct workflow for political speech: classify it, escalate to a human reviewer, and never auto-remove it under any circumstances.

Few-shot examples improve compliance rates but cannot guarantee 100% adherence. Platform policy requires deterministic enforcement, not improved probability.

C A PreToolUse hook that checks the category before any removal runs and blocks it for 'political_speech', redirecting to human review.

A PreToolUse hook provides deterministic enforcement by physically blocking the removal tool when the content category is political speech. This is the only approach that guarantees 100% compliance with the policy, which is what platform rules require.

WHERE THIS COMES FROM

Lesson 1.5: Agent SDK Hooks (PreToolUse hooks)

Anthropic: Agent SDK Hooks ↗

Q18 · D1 1.2 ORCHESTRATION-PATTERNS / ISOLATION q-1-2-009 · Content Moderation and Classification System CORRECT

The content moderation system uses a hub-and-spoke architecture with a coordinator that routes posts to specialist subagents: a text classifier, an image analyser, and a policy enforcer. The team notices that the image analyser sometimes directly calls the policy enforcer's action tools to remove posts, bypassing the coordinator. What is the architectural violation and how should it be fixed?

C Merge the image analyser and policy enforcer into a single subagent to simplify the communication flow

Merging subagents with distinct responsibilities (analysis vs enforcement) violates separation of concerns. The image analyser and policy enforcer have fundamentally different roles and tool requirements.

A Give the image analyser its own copy of the policy enforcer's action tools so it can remove posts itself without calling another subagent.

Duplicating action tools across subagents creates inconsistent enforcement and makes policy changes harder to propagate. The issue is the communication pattern, not tool availability.

D Add a message queue between the image analyser and the policy enforcer so communication is asynchronous

An async queue between subagents still bypasses the coordinator. In hub-and-spoke, all inter-agent communication must flow through the coordinator regardless of synchronicity.

B The image analyser breaks hub-and-spoke isolation; scope its tools to image analysis and route all results back through the coordinator.

In hub-and-spoke orchestration, subagents must be isolated and communicate only through the coordinator. The image analyser should return its classification result to the coordinator, which then decides whether to invoke the policy enforcer. Direct subagent-to-subagent communication breaks the architecture.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Isolation principle)

Lesson 1.2: Multi-Agent Orchestration (Hub-and-spoke architecture)

Q19 · D1 1.2 ORCHESTRATION-PATTERNS / COORDINATOR q-1-2-013 · Multi-Agent Research System

CORRECT

In a hub-and-spoke research system, a coordinator manages several search and analysis subagents. Which responsibilities sit with the coordinator rather than with individual subagents? (Select 3)

E Aggregating subagent results and re-delegating targeted follow-up queries when synthesis shows gaps.

Result aggregation and gap-driven re-delegation are coordinator responsibilities in the iterative refinement loop.

D Running every search itself first so subagents only verify its findings.

The coordinator delegates work rather than duplicating it; doing every search itself defeats the point of decomposition.

B Deciding which subagents to invoke based on the complexity of each incoming query.

The guide places dynamic subagent selection with the coordinator, which analyses query requirements instead of always running the full pipeline.

C Routing all inter-agent communication so subagents never exchange messages directly.

Hub-and-spoke routes every message through the coordinator for observability and consistent error handling.

A Sharing its full conversation history with each subagent automatically at spawn time.

Subagents run with isolated context; nothing is inherited automatically, and the coordinator passes only the context each task needs.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Lesson 1.2: Multi-Agent Orchestration (Hub-and-spoke architecture)

Q20 · D3 3.6 CI/CD-INTEGRATION / OUTPUT-FORMAT q-3-6-004 · CI/CD Pipeline Integration

CORRECT

A team wants their CI pipeline to run Claude Code for both PR code review and automated test generation. The PR review should output structured JSON for integration with their review dashboard, while the test generation should produce standard text output. How should the two pipeline steps be configured?

C Run both steps in a single Claude Code session with -p, using different prompts to request different output formats

Running both steps in a single session violates session context isolation. The review step would retain reasoning context from test generation, reducing review effectiveness. Each step should be an independent invocation.

D Run both steps with `--output-format json` and have the test generation step extract code from the JSON response

While this would work technically, it adds unnecessary complexity to the test generation step. Using JSON output format only for the step that needs structured parsing (review) is the cleaner approach.

B Run the review step with `-p --output-format json` and the test generation step with `-p` only, as separate non-interactive invocations

Each step uses `-p` for non-interactive mode. The review step adds `--output-format json` for structured output that the dashboard can parse reliably. The test generation step uses default text output since it produces code files, not structured data. Running them as separate invocations ensures session context isolation.

A Run both steps with `-p` and configure the review dashboard to parse plain text output from both steps

Plain text output requires fragile parsing logic in the review dashboard. The `--output-format json` flag provides structured, machine-readable output that is far more reliable for programmatic integration.

WHERE THIS COMES FROM

Lesson 3.6: CI/CD Integration (Structured output)

Lesson 3.6: CI/CD Integration (-p flag)

Claude Code: Headless / CLI Reference ↗

Q21 · D5 5.3 ERROR-PROPAGATION / COVERAGE-ANNOTATIONS q-5-3-007 · Technical Documentation Maintenance System **CORRECT**

A documentation generation pipeline uses Claude Code to produce docs from three sources: source code comments, existing wiki pages, and API schema files. During a run, the wiki page retrieval fails with a timeout, but the source code and API schema are available. The pipeline currently halts entirely on any source failure. What is the best error handling strategy?

D Use the source code comments to infer what the wiki pages would have contained, filling in the gaps with generated content

Inferring wiki content from source code produces hallucinated documentation that appears authoritative. Wiki pages often contain operational context, known issues, and tribal knowledge that cannot be inferred from code.

A Retry the wiki retrieval three times with exponential backoff. If all retries fail, halt the pipeline to prevent incomplete documentation

Halting the entire pipeline because one of three sources is unavailable wastes the successful results from the other two sources. Documentation from source code and API schemas is still valuable even without wiki content.

B Return structured error context for the wiki failure, proceed with available sources, and mark the gaps that lack wiki content.

This produces maximum value from available sources while maintaining transparency about what is missing. Structured error context enables intelligent recovery (retry later, alert the team). Explicit gap markers prevent users from trusting incomplete documentation as complete.

C Silently skip the wiki source and generate documentation from the remaining two sources without noting the omission

Silent omission hides the failure. Users would trust the documentation as complete, not knowing that wiki-sourced context (which may contain critical operational notes or caveats) is missing.

WHERE THIS COMES FROM

Lesson 5.3: Error Propagation in Multi-Agent Systems (Structured error context)

Lesson 5.3: Error Propagation in Multi-Agent Systems (Coverage annotations)

Q22 · D4 4.3 STRUCTURED-OUTPUT / ENUM-FIELDS q-4-3-004 · Content Moderation and Classification System **CORRECT**

The moderation system's classification schema has a 'category' field defined as a free-text string. Auditors find 47 different category values in production data, including 'hate speech', 'Hate Speech', 'hate-speech', 'hateful content', and 'hate_speech' — all intended to be the same category. This makes downstream analytics and routing unreliable. What is the best schema fix?

C Add detailed instructions to the prompt listing the exact category names and their capitalisation so the model always emits the canonical string.

Prompt instructions are probabilistic. The model may still produce variations despite instructions. Schema-level enforcement via enums is deterministic and cannot be overridden.

A Add a post-processing normalisation step that maps all variations to canonical category names

Post-processing normalisation adds complexity and requires constant maintenance as new variations appear. The schema should prevent the problem at the source rather than patching it downstream.

D Add few-shot examples showing the correct category formatting for each type

Few-shot examples improve consistency but cannot guarantee exact string matching. Schema enums are the correct mechanism for constraining categorical output to a fixed set of values.

B Change 'category' from free-text to an enum with values like 'hate_speech', 'spam', and 'harassment', plus an 'other' option.

Enum fields constrain the model to predefined values, eliminating spelling and formatting variations. Including 'other' as an enum value handles edge cases without allowing free-text drift. With strict: true, the schema guarantees only valid enum values are returned.

WHERE THIS COMES FROM

Lesson 4.2: Structured Output with Tool Use (Enum schema)

Q23 · D4 4.6 MULTI-PASS-REVIEW / MULTI-PASS q-4-6-004 · CI/CD Pipeline Integration

CORRECT

Your CI/CD code review system analyses pull requests averaging 8-12 files. Reviews are thorough on the first few files but become increasingly superficial on later files, sometimes missing obvious issues. What architectural change best addresses this pattern?

C Split the review into per-file passes so each file gets dedicated attention, then a cross-file integration pass.

Per-file analysis ensures each file receives consistent, thorough attention. The cross-file integration pass then catches issues that span multiple files, such as inconsistent interfaces or data flow problems. This directly solves attention dilution.

B Increase the model's context window to give it more space to analyse all files

Attention dilution is not a context window size problem. The model has enough space but distributes attention unevenly across many files.

D Add a second full-PR review pass and merge findings from both passes

A second pass on the full PR suffers from the same attention dilution. Both passes will likely be thorough on early files and superficial on later ones.

A Randomise the file order before each review run so a different subset of files receives the thorough early-pass attention each time.

Randomising order means different files get superficial treatment each time, but does not ensure all files receive thorough review. The root cause — attention dilution — remains.

WHERE THIS COMES FROM

Lesson 4.6: Multi-Instance Review and Output Validation (Multi-pass review)

Lesson 4.6: Multi-Instance Review and Output Validation (Why context windows do not fix it)

Q24 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / PARALLEL-SPAWNING q-1-3-010 · Content Moderation and Classification System

CORRECT

A user reports a post that contains both potentially defamatory text and an embedded image that may violate graphic content policies. The coordinator receives this report as a single moderation request. What is the correct delegation strategy?

A Send the entire report to the text classifier first, then forward its output to the image analyser so the image step runs only after the text step has finished.

Sequential processing creates unnecessary latency when the two concerns are independent. The text and image analyses do not depend on each other and can run in parallel.

D Create a new combined text-and-image subagent specifically for multi-modal reports

Creating a new combined subagent duplicates capability that already exists in the specialist subagents. The coordinator's role is to decompose and delegate, not to spawn new agents for every combination of concerns.

B Send the entire report to whichever subagent handles the more severe category

Severity cannot be determined until both analyses are complete. Routing to a single subagent also means one concern goes unanalysed.

C Route the text to the text classifier and the image to the image analyser in parallel, then aggregate both results for the final decision.

The coordinator should decompose multi-concern requests into independent subtasks. Text defamation and image policy violations are separate analysis dimensions that can be evaluated in parallel. The coordinator aggregates the results to make a unified moderation decision.

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Parallel spawning)

Lesson 1.4: Workflow Enforcement and Handoff (Multi-concern request handling)

Q25 · D3 3.1 CLAUDE-MD-HIERARCHY / PRECOMPACT-HOOK q-3-1-018 · Technical Documentation Maintenance System

CORRECT

A documentation team wants to archive the full conversation transcript to a log file every time Claude Code runs `/compact`, so that context lost during compaction can be reviewed later. Which hook configuration achieves this?

B A PreCompact hook that writes the current transcript to a timestamped log file before `/compact` summarises the conversation

PreCompact is a first-class hook event in Claude Code that fires immediately before `/compact` (or auto-compaction) runs. The hook receives the full pre-compaction transcript path and can copy or process it before summarisation discards detail. This is the documented mechanism for exactly this use case.

C A PreToolUse hook configured on a built-in 'Compact' tool, matching tool name 'Compact'

There is no 'Compact' tool exposed to PreToolUse. Compaction is a Claude Code lifecycle operation, not a model-invoked tool, so it has its own dedicated PreCompact event rather than being hooked via PreToolUse.

A A PostToolUse hook on Write that snapshots the transcript whenever a file is written, on the assumption `/compact` will eventually run

Write and compaction are unrelated. PostToolUse on Write would fire on every file write whether or not `/compact` is about to run, producing noise rather than a transcript at the right moment. The dedicated PreCompact event is the correct hook.

D A PostToolUse hook on all tools that appends each tool result to a running log file, creating a continuous archive

Logging every tool result creates enormous log files and does not capture conversation context (reasoning, planning, user messages). The targeted PreCompact event captures the full transcript at the critical moment without per-tool noise.

WHERE THIS COMES FROM

Claude Code: Hooks ↗

Q26 · D1 1.2 ORCHESTRATION-PATTERNS / ISOLATION q-1-2-010 · Content Moderation and Classification System

INCORRECT

The moderation coordinator dispatches posts to specialist subagents. The text classifier has been extended to also handle image analysis: its prompt is now 2,500 tokens, its tool list has grown to 12 tools, and accuracy has dropped on both text and image tasks. What principle was violated and what is the fix?

A The text classifier's context window is too small — upgrade to a model with a larger context window to handle the expanded prompt

Context window size is not the issue. Combining unrelated responsibilities in a single agent degrades performance because the model must juggle competing instructions and tools. A larger context window does not fix scope creep.

C Add better instructions to the combined subagent's prompt to clarify when it should use text tools versus image tools

Better instructions do not address the fundamental problem of an overloaded agent. Twelve tools and 2,500 tokens of instructions degrade performance regardless of instruction quality. The fix is architectural, not instructional.

D Split the combined agent into three narrower specialists: text sentiment, text policy, and image analysis

Over-splitting into too many narrow specialists adds coordination overhead without clear benefit. The original architecture had a text classifier and an image analyser, which is the correct level of decomposition. Restoring that structure is the fix. (your answer)

B Isolation was violated; restore the text classifier to text-only work with scoped tools and keep image analysis separate.

Each subagent should have a focused responsibility and a scoped tool set (4-5 tools maximum). Combining text and image analysis in one agent violates isolation, bloats the prompt, and confuses the model. Restoring separate specialists with scoped tools improves accuracy for both tasks. (correct answer)

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Isolation principle)

Lesson 1.4: Workflow Enforcement and Handoff (Scoped tool access)

Q27 · D4 4.6 MULTI-PASS-REVIEW / MULTI-PASS q-4-6-001 · CI/CD Pipeline Integration

CORRECT

A PR modifying 14 files receives inconsistent review: detailed feedback on some files, superficial comments on others, and contradictory findings (flagging a pattern in one file while approving identical code elsewhere). What is the best restructuring?

B Run three independent passes on the full PR and only flag issues found in at least two passes

This suppresses real bug detection by requiring consensus on issues that may only be caught intermittently.

D Require developers to split large PRs into smaller submissions of 3-4 files

This shifts burden to developers without improving the review system itself.

A Switch to a higher-tier model with a larger context window

Larger context windows do not solve attention quality issues. The problem is attention dilution, not context size.

C Split into per-file local analysis passes plus a separate cross-file integration pass

Per-file analysis ensures consistent depth. The integration pass catches cross-file data flow issues. This directly addresses attention dilution.

WHERE THIS COMES FROM

Lesson 4.6: Multi-Instance Review and Output Validation (Multi-pass review)

Q28 · D3 3.6 CI/CD-INTEGRATION / P-FLAG q-3-6-001 · CI/CD Pipeline Integration

CORRECT

A CI pipeline script runs 'claude "Analyse this PR"' but the job hangs indefinitely. Logs show Claude Code is waiting for interactive input. What is the correct fix?

D Add the --batch flag: claude --batch "Analyse this PR"

--batch is not a real Claude Code CLI flag. This option references a non-existent feature.

A Add the -p flag: claude -p "Analyse this PR"

The -p (--print) flag is the documented way to run Claude Code in non-interactive mode. It processes the prompt, outputs the result to stdout, and exits without waiting for user input.

C Redirect stdin from /dev/null: claude "Analyse this PR" < /dev/null

Unix stdin redirection does not properly address Claude Code's interactive mode. The -p flag is the correct, documented approach.

B Set the environment variable CLAUDE_HEADLESS=true before running the command

CLAUDE_HEADLESS is not a real Claude Code environment variable. This option references a non-existent feature.

WHERE THIS COMES FROM

Lesson 3.6: CI/CD Integration (-p flag)

Claude Code: Headless / CLI Reference ↗

Q29 · D4 · 4.1 SYSTEM-PROMPTS / FALSE-POSITIVES · q-4-1-001 · CI/CD Pipeline Integration

CORRECT

Your CI/CD code review pipeline has a 40% false positive rate on 'documentation mismatch' findings, causing developers to ignore ALL review categories. What is the most effective fix?

D Add a second model pass to verify each finding before reporting

A second pass without better criteria will have the same false positive problem. Fix the criteria first.

A Add 'only report high-confidence documentation issues' to the system prompt

Vague confidence instructions do not improve precision. The model has no concrete criteria for what 'high-confidence' means.

C Increase the model temperature to get more varied results and filter outliers

Temperature affects randomness, not precision. This would likely increase false positives.

B Temporarily disable the documentation mismatch category while refining its prompts with explicit criteria and code examples

This restores trust in other categories immediately while you iterate on the problematic category with specific criteria.

WHERE THIS COMES FROM

Lesson 4.1: System Prompts with Explicit Criteria (False-positive trust problem)

Q30 · D1 · 1.1 AGENTIC-LOOPS / MODEL-DRIVEN · q-1-1-008 · Content Moderation and Classification System

INCORRECT

The moderation system's agentic loop uses a hardcoded decision tree: if the `classify_content` tool returns 'hate_speech', always call `escalate_to_human`; if it returns 'spam', always call `auto_remove`. During testing, the team discovers that satirical posts criticising hate speech are being auto-escalated, and sophisticated spam disguised as legitimate marketing slips through. What is the architectural problem?

B Replace the hardcoded decision tree with model-driven decisions, letting Claude weigh the full context of each post before it acts.

Hardcoded decision trees treat classification labels as absolute when real content is nuanced. Letting Claude reason about context (satire vs genuine hate, sophisticated spam patterns) produces better moderation decisions. The agentic loop should let the model decide, not map labels to fixed actions. (correct answer)

A The `classify_content` tool needs more granular category labels so it can tell satire criticising hate speech apart from genuine hate speech.

More granular labels do not address the core issue. A hardcoded decision tree cannot adapt to nuance, such as satire versus genuine hate, no matter how fine the labels are. The fix is model-driven reasoning. (your answer)

D Route all ambiguous cases to human review to avoid misclassification

Routing everything ambiguous to humans defeats the purpose of automated moderation. The model can reason about context effectively when given the opportunity; the hardcoded tree is what prevents it.

C Add a confidence threshold so only high-confidence classifications trigger automatic actions

Confidence thresholds reduce false positives but the fundamental problem remains: a hardcoded tree cannot handle contextual nuance. Low-confidence cases still need model-driven reasoning, not just deferral.

WHERE THIS COMES FROM

Lesson 1.1: Agentic Loops (Model-driven decision-making)

Lesson 1.1: Agentic Loops (Anti-patterns)

Q31 · D4 · 4.3 STRUCTURED-OUTPUT / NULLABLE-FIELDS · q-4-3-003 · Content Moderation and Classification System

CORRECT

The moderation system uses tool_use with a JSON schema for classification output. All fields including 'sub_category' and 'target_demographic' are marked as required. Auditors discover that when a post is spam (which has no target demographic), the model fabricates plausible-sounding demographics like 'general public' or 'young adults.' What schema change prevents this fabrication?

C Remove 'target_demographic' from the schema entirely since it causes fabrication

Removing the field loses valuable data for categories where target demographic is genuinely relevant (hate speech, harassment). The fix is making it nullable, not removing it.

A Add a validation step that rejects target_demographic values for spam posts

Post-hoc validation catches the problem but does not prevent it. The model still fabricates values; they are just rejected after the fact. Schema design should prevent fabrication at the source.

D Add a prompt instruction telling the model to leave 'target_demographic' empty whenever a post is spam and the field does not apply.

Prompt instructions cannot override a schema that marks the field as required. The JSON schema enforces the constraint at the API level, and the model must comply with the schema regardless of prompt instructions.

B Make 'target_demographic' nullable so the model can return null when the field does not apply instead of fabricating a value.

Required fields pressure the model to produce a value even when none exists. Making the field nullable gives the model a legitimate way to indicate 'not applicable,' eliminating the incentive to fabricate. This is the standard pattern for fields that apply conditionally.

WHERE THIS COMES FROM

Lesson 4.2: Structured Output with Tool Use (Nullable fields)

Q32 · D3 · 3.4 PLAN-MODE-EXECUTION / PLAN-MODE · q-3-4-008 · Technical Documentation Maintenance System

INCORRECT

A documentation team must update architecture guides across 25 microservice directories after a platform migration. Each guide requires reading source, checking dependencies, and rewriting affected sections. Working in direct execution mode, a team member updates three services before noticing the results are inconsistent: some guides still mention the old platform, others use inconsistent terminology for the new one. What went wrong and what is the correct approach?

D The team member should have used the Explore subagent to discover all affected files first, then made all changes in a single batch

Discovery alone does not solve the consistency problem. Without an explicit plan defining terminology and update patterns, batch execution would produce the same inconsistencies faster. (your answer)

B Direct execution was correct but should have been done in a single continuous session to maintain consistency through conversation context

A single session updating 25 services would exhaust the context window, causing later updates to lose awareness of earlier decisions. Context degradation would worsen the inconsistency problem, not solve it.

C The team member should have created a custom slash command that hardcodes the exact text replacements for each service

Architecture guides require contextual understanding of each service's code and dependencies — not mechanical text replacement. A slash command cannot adapt to the unique structure of each guide.

A The team member should have used plan mode to set a consistent update strategy and terminology before executing across all 25 services

A 25-service migration with consistency requirements is a multi-step, cross-file operation where planning is essential. Plan mode would produce an explicit strategy covering terminology, update order, and validation criteria before any changes are made, preventing the inconsistency that emerged from ad hoc execution. (correct answer)

WHERE THIS COMES FROM

Lesson 3.4: Plan Mode vs Direct Execution (Plan mode for consistency)

Lesson 3.4: Plan Mode vs Direct Execution (Decision framework)

Q33 · D2 2.2 STRUCTURED-ERROR-RESPONSES / ERROR-CONTEXT q-2-2-011 · Multi-Agent Research System CORRECT

A research subagent's database query fails partway through an investigation. Which pieces of information belong in the structured error context it returns to the coordinator? (Select 3)

B Partial results gathered before the failure.

Partial results preserve completed work so the coordinator does not re-run what already succeeded.

C The failure type, such as transient, validation, or permission.

Categorising the failure lets the coordinator make the right retry-or-reroute decision.

E A plain 'no results found' message in place of the error.

Reporting an access failure as an empty result conflates two different situations and hides the need for a retry decision.

A No report at all: the subagent should retry silently until the query succeeds.

Silent retries hide failures from the coordinator, which owns error handling and needs observability into what went wrong.

D What was attempted, including the query that failed.

The attempted query gives the coordinator enough detail to retry intelligently or reformulate the request.

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Four error categories)

Lesson 2.2: Structured Error Responses (Access failure vs empty result)

MCP: Tools Specification ↗

Q34 · D4 4.1 SYSTEM-PROMPTS / EXPLICIT-CRITERIA q-4-1-006 · Content Moderation and Classification System CORRECT

The moderation system's prompt instructs Claude to 'be conservative when moderating and err on the side of caution.' Reviewers find that innocuous posts about cooking with knives, news articles about violence, and fictional war stories are all being flagged as policy violations. What is the root cause and fix?

A The model is too sensitive — lower the temperature to reduce over-flagging

Temperature affects randomness, not the interpretation of moderation criteria. The over-flagging is caused by vague instructions, not sampling behaviour.

B Replace 'be conservative' with explicit categorical criteria defining each violation category with concrete examples.

Vague directives like 'be conservative' give the model no basis for distinguishing between genuine violations and legitimate content that mentions sensitive topics. Explicit criteria with examples of both violations and non-violations (news, fiction, educational content) enable consistent, calibrated decisions.

C Add an allowlist of safe topics (cooking, news, fiction) that should never be flagged

Allowlists are brittle and impossible to maintain comprehensively. New safe topics will constantly appear. The fix is better criteria that teach the model to distinguish context, not a growing list of exceptions.

D Add a second moderation pass that re-reads each flagged post under the same 'be conservative' guidance and strips out the false positives before any action.

A second pass using the same vague 'be conservative' criteria will reproduce the same false positives. Fix the criteria first; add verification layers second.

WHERE THIS COMES FROM

Lesson 4.1: System Prompts with Explicit Criteria (Explicit criteria over hedge phrases)

Lesson 4.1: System Prompts with Explicit Criteria (Severity calibration)

Q35 · D2 2.5 BUILT-IN-TOOLS / EDIT-REPLACE-ALL q-2-5-004 INCORRECT

A developer needs to rename a variable from 'userData' to 'customerData' in a specific file. The variable appears 12 times throughout the file. What is the most efficient approach using Claude Code's built-in tools?

A Use Grep to find all 12 occurrences, then call Edit 12 times, once for each occurrence with unique surrounding context.

Calling Edit 12 times is inefficient. The Edit tool's `replace_all` parameter is designed for exactly this use case — renaming a variable across an entire file in a single call. (your answer)

D Use Bash with `sed` to perform a global find-and-replace across the file.

While `sed` can perform the replacement, the exam expects candidates to use built-in tools for file modifications. Edit with `replace_all` is the purpose-built tool for this task.

C Use Edit with `replace_all` set to true, specifying 'userData' as the old string and 'customerData' as the new string.

The Edit tool's `replace_all` parameter replaces all occurrences of the specified string in a single operation. This is the most efficient approach for renaming a variable throughout a file. (correct answer)

B Use Read to load the file, then Write the entire file back with all occurrences manually changed.

Read + Write rewrites the entire file, which is wasteful for a simple rename. The Edit tool with `replace_all` handles this more efficiently by sending only the diff.

WHERE THIS COMES FROM

Lesson 2.5: Built-in Tools (Edit `replace_all`)

Claude Code: Built-in Tools ↗

Q36 · D5 5.1 CONTEXT-WINDOW-MANAGEMENT / LOST-IN-THE-MIDDLE q-5-1-008 ·

Technical Documentation Maintenance System

CORRECT

Claude Code is synthesising release notes from commit messages, PR descriptions, and changelog entries. During a 200+ commit session, summaries of early commits become vague ('various bug fixes') while recent commits are still detailed accurately. The context window is not full. What is the most likely cause?

A The model's temperature is set too high, causing it to generate vague summaries randomly

Temperature affects output randomness but would produce inconsistent quality across all commits, not a systematic pattern where early commits are vague and recent commits are accurate.

B The lost-in-the-middle effect: the model favours the start and end of long input, losing middle commit detail.

The lost-in-the-middle effect is a well-documented phenomenon where models attend more strongly to the start and end of long contexts, with reduced attention to middle content. With 200+ commits loaded sequentially, commits in the middle of the sequence receive less attention, producing vague summaries. Processing in smaller batches or reordering critical information to the start and end mitigates this.

C The commit messages for early commits are inherently less detailed than recent ones, so the vague summaries are accurate

The question states the vagueness applies to early commits specifically, not commits with poor messages. The systematic pattern (early = vague, recent = detailed) points to a context positioning effect, not source quality variation.

D Claude Code applies progressive summarisation to older commits to conserve context for recent ones

Claude Code does not automatically apply progressive summarisation to tool results within a single processing step. The context window is not full, so there is no space pressure triggering summarisation. The issue is attention distribution across long inputs.

WHERE THIS COMES FROM

Lesson 5.1: Context Window Management (Lost in the middle)

Q37 · D5 5.4 CODEBASE-EXPLORATION / CONTEXT-DEGRADATION q-5-4-008 ·

Technical Documentation Maintenance System

CORRECT

A documentation team working across a 500,000-line codebase notices that Claude Code's responses become less specific after extensive file exploration. What is the term for this degradation and what is the primary mitigation in Claude Code?

A Token exhaustion, mitigated by raising max_tokens so the model has room to keep referencing every file it has read.

max_tokens controls response length, not the model's ability to retain and reference earlier context. The degradation occurs because verbose exploration output fills the context window, not because responses are truncated.

B Context degradation, mitigated by scratchpad files and by delegating verbose exploration to subagents.

Context degradation occurs when the model loses grip on specific details as verbose tool output accumulates in the conversation. Scratchpad files persist findings on disk where they can be re-read as needed. Subagent delegation isolates verbose exploration output from the main agent's context budget.

C Hallucination drift — mitigated by lowering the temperature to zero for deterministic output

The issue is not hallucination from randomness but loss of specificity from context overload. Temperature zero produces deterministic output but does not help the model retain specific findings from earlier in a long context.

D Prompt injection — mitigated by sanitising file contents before they enter the context window

Prompt injection is a security concern involving malicious content in inputs. The described problem — responses becoming vague after extensive exploration — is context degradation from accumulated verbose output, not a security issue.

WHERE THIS COMES FROM

Lesson 5.4: Codebase Exploration and Context Degradation (Context degradation)

Lesson 5.4: Codebase Exploration and Context Degradation (Scratchpad mitigation)

Q38 · D5 5.1 CONTEXT-WINDOW-MANAGEMENT / PROGRESSIVE-SUMMARISATION q-5-1-009 ·

Technical Documentation Maintenance System

CORRECT

A docs team processes a 500,000-line codebase by having Claude Code read files and generate docs, instructing it to 'summarise each module after documenting it, then discard the detailed source from context.' After processing 20 modules, the docs for module 21 reference class names from module 3 that were renamed in module 12. What caused this failure?

B The agent's context window overflowed, causing it to hallucinate class names from earlier modules

The team explicitly manages context by summarising and discarding source code, so the context window is not overflowing. The problem is the quality of information retained in summaries, not context overflow.

D The model confused similar class names because it processed too many modules in a single session

The question specifies the exact failure: module 21 references class names from module 3 that were renamed in module 12. This is not a confusion issue but a direct consequence of losing the rename detail during progressive summarisation.

C Claude Code cached module 3's source code and served the stale version instead of the updated file on disk

Claude Code does not cache file contents between reads. Each Read tool call fetches the current file from disk. The issue is that summarised context retained old class names, not that file reads returned stale data.

A Progressive summarisation lost the rename detail: when module 12's summary replaced its source code, the rename was condensed to 'various refactoring changes', so the agent retained module 3's original class names from its earlier summary

Progressive summarisation is lossy. The summary of module 12 likely compressed the rename into a generic phrase, while module 3's summary still referenced the original class names. When the agent documented module 21, it found conflicting names across summaries and defaulted to the earlier, more prominent reference. Critical cross-module changes like renames must be tracked in a persistent facts block outside summarised context.

WHERE THIS COMES FROM

Lesson 5.1: Context Window Management (Progressive summarisation)

Lesson 5.1: Context Window Management (Persistent facts)

Q39 · D1 1.7 SESSION-STATE-RESUMPTION / FRESH-START q-1-7-001 · Multi-Agent Research System

CORRECT

A developer resumes a Claude Code session after modifying 3 files in a 50-file codebase. The agent gives contradictory advice about the modified files because it is reasoning from stale tool results. What is the correct approach?

A Start a completely new session and have the agent re-analyse all 50 files from scratch so nothing stale survives into the new context.

Re-analysing all 50 files is wasteful when only 3 changed. A fresh session with summary injection plus targeted re-reading of the 3 changed files is more efficient.

B Resume the session and ask the agent to re-read only the 3 modified files

Resuming preserves the stale tool results in context. The agent may still reason from outdated information for other decisions. A fresh start with summary injection is more reliable.

D Use `fork_session` to create a branch that incorporates the file changes

`fork_session` creates an independent exploration branch from a shared baseline. It is for divergent approaches, not for updating stale context after file changes.

C Start a fresh session with an injected summary of prior findings, noting the 3 changed files for targeted re-analysis.

A fresh session with summary injection avoids stale tool results. Specifying the 3 changed files enables targeted re-analysis without wasting time on unchanged files.

WHERE THIS COMES FROM

Lesson 1.7: Session State and Resumption (Stale context)

Lesson 1.7: Session State and Resumption (Targeted re-analysis)

Anthropic: Claude Code Documentation ↗

Q40 · D4 4.5 BATCH-PROCESSING / BATCH-VS-REALTIME q-4-5-001 · CI/CD Pipeline Integration

CORRECT

Your manager proposes switching both your blocking pre-merge code review and your overnight technical debt report to the Message Batches API for 50% cost savings. How should you evaluate this proposal?

B Use batch processing for the technical debt reports only; keep real-time calls for pre-merge checks

Pre-merge checks are blocking workflows where developers wait. The batch API's 24-hour window makes it unsuitable. Technical debt reports are overnight and latency-tolerant.

C Keep real-time calls for both to avoid batch result ordering issues

Batch results can be correlated using `custom_id` fields. Ordering is not the issue — latency requirements are.

A Switch both to batch processing with status polling to check for completion

Batch processing has no guaranteed latency SLA. Polling does not make it suitable for blocking pre-merge checks.

D Switch both to batch with a timeout fallback to real-time if batches take too long

This adds unnecessary complexity. Match each API to its latency requirements: synchronous for blocking, batch for tolerant.

WHERE THIS COMES FROM

Lesson 4.5: Batch Processing and Prompt Optimisation (Batches API facts)

Lesson 4.5: Batch Processing and Prompt Optimisation (SLA implications)

Anthropic: Message Batches API ↗

Q41 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / SCOPED-TOOLS q-1-3-017 · Multi-Agent Research System

CORRECT

A web search agent has 9 tools: `'web_search'`, `'url_fetch'`, `'html_parse'`, `'pdf_extract'`, `'image_ocr'`, `'translate'`, `'summarise'`, `'keyword_extract'`, `'sentiment_analysis'`. In testing it frequently calls `'summarise'` and `'sentiment_analysis'` when it should only fetch raw data. How should the architect fix this?

D Merge the web search and synthesis agents into a single agent since they share some tools

Merging agents violates the principle of specialised subagents. The solution is to separate tools by role, not to combine agents. Merging would create an even larger tool set on a single agent.

A Add system prompt instructions telling the web search agent to only use data fetching tools and ignore analysis tools

Prompt instructions are probabilistic and the agent is already misusing tools. The architectural fix is to remove the tools it should not have, providing deterministic enforcement through scoped tool access.

B Reduce the web search agent to the 4-5 data-fetching tools and move the analysis tools to the specialist agents.

The recommended maximum is 4–5 tools per agent. The web search agent has 9 tools, causing it to use tools outside its intended role. Reducing to data-fetching tools enforces scoped access. The analysis tools (summarise, sentiment_analysis, keyword_extract) belong with the synthesis or analysis agents.

C Keep all 9 tools but implement PreToolUse hooks that block the web search agent from calling summarise and sentiment_analysis

Using hooks to block tool access is an over-engineered workaround for what should be a configuration fix. The proper solution is to remove the tools from allowedTools, not to provide them and then block their use.

WHERE THIS COMES FROM

Lesson 1.4: Workflow Enforcement and Handoff (Scoped tool access (4-5 tools))
Anthropic: Claude Agent SDK Overview ↗

Q42 · D4 4.5 BATCH-PROCESSING / PROMPT-CACHING q-4-5-006 · CI/CD Pipeline Integration CORRECT

Your Claude Code setup uses a complex system prompt with detailed review instructions, code examples, and few-shot demonstrations that total 4,000 tokens. Each code review request adds 500-2,000 tokens of code to review. The team wants to optimise costs since the same review instructions are used for every request. What is the most effective optimisation?

A Shorten the system prompt by removing few-shot examples to reduce per-request token costs

Removing few-shot examples degrades review quality. The 4,000-token static prompt is the ideal candidate for caching, not trimming.

B Enable prompt caching with the static review instructions and examples first and the code-to-review at the end.

Prompt caching caches the static prefix and reuses it across requests. With 4,000 tokens of static content reused for every review, caching provides significant cost savings without sacrificing review quality. Dynamic code goes at the end so the prefix remains cacheable.

D Reduce the number of few-shot examples from 4 to 1 and lower the temperature to compensate for the reduced guidance

Lower temperature does not compensate for reduced examples. Fewer examples degrade quality. Prompt caching preserves the full prompt while reducing costs.

C Switch all code reviews to the Message Batches API for 50% cost savings

Code reviews in Claude Code are interactive, blocking workflows. The Batches API's 24-hour window makes it unsuitable for developer-facing reviews.

WHERE THIS COMES FROM

Lesson 4.5: Batch Processing and Prompt Optimisation (Prompt caching layout)
Anthropic: Prompt Caching ↗

Q43 · D1 1.2 ORCHESTRATION-PATTERNS / COORDINATOR q-1-2-005 · Multi-Agent Research System CORRECT

A research coordinator spawns 4 subagents to investigate different aspects of a market analysis. Agent A fails with an API error, Agent B returns partial results (3 of 5 requested data points), and Agents C and D return complete results. The final report is due in 2 hours. What should the coordinator do?

A Retry Agent A, discard Agent B's partial results as unreliable, and wait for both agents to return fully complete data before writing the report.

Discarding Agent B's partial results wastes valid data, and waiting for full completion ignores the two-hour deadline. The coordinator should use what is available while attempting to fill gaps.

C Retry Agent A, incorporate Agent B's partial results, flag B's missing data points as gaps, and synthesise from all available results.

The coordinator should maximise data utilisation: retry the failed agent (A), incorporate partial results from B while flagging the 2 missing data points as known gaps, and synthesise all available results from A (after retry), B (partial), C, and D. Explicit gap annotations ensure the report is transparent about its completeness.

B Proceed with only Agents C and D's results and produce the report without the data from Agents A and B

This unnecessarily discards Agent B's partial results and abandons Agent A's scope entirely. A thorough coordinator should attempt recovery while incorporating all available data.

D Terminate all agents and restart the entire investigation from scratch with a simplified scope

Restarting discards the complete results from Agents C and D and the partial results from Agent B. This wastes time and compute when most of the investigation has already succeeded.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Lesson 1.7: Session State and Resumption (Recovering from partial results)

Q44 · D5 5.3 ERROR-PROPAGATION / ACCESS-VS-EMPTY q-5-3-002 · Multi-Agent Research System **CORRECT**

A research subagent queries an academic journal database for articles on renewable energy policy. The database responds successfully but returns zero matching articles. Meanwhile, a second subagent querying a government statistics API receives a connection timeout. The coordinator currently handles both cases identically by retrying three times. What should change?

B Add exponential backoff to all retries so the subagents wait progressively longer between attempts

Backoff improves retry behaviour for transient failures but does not address the fundamental issue: the journal query does not need retries at all. It returned a valid response indicating no matches exist.

C Remove retries entirely and immediately escalate both failures to a human operator for investigation

This is disproportionate. The API timeout is a transient failure that retries can resolve. The empty journal result is not a failure at all — it is a valid answer. Neither case requires human escalation.

D Distinguish access failures from valid empty results: retry the API timeout as a transient failure, but accept the zero-match journal response as a valid finding and annotate the coverage gap

Access failures (timeouts, connection errors) indicate the tool could not reach the data source and warrant retry. Valid empty results mean the tool reached the source and found no matches — this IS the answer. Conflating these leads to unnecessary retries on valid results and missed coverage annotations.

A Increase the retry count to five for both cases to give transient failures more time to resolve

More retries do not fix the core problem. The journal database returned a valid empty result — it successfully searched and found nothing. Retrying it wastes time and API calls on an operation that already completed correctly.

WHERE THIS COMES FROM

Lesson 5.3: Error Propagation in Multi-Agent Systems (Access failure vs empty)

Lesson 5.3: Error Propagation in Multi-Agent Systems (Local recovery for transient)

Q45 · D5 5.3 ERROR-PROPAGATION / SILENT-SUPPRESSION q-5-3-005 · Multi-Agent Research System **CORRECT**

A multi-agent system has subagents for web search, academic database, and industry reports. The academic database subagent hits a 403 Forbidden on a specific journal, catches the error, and returns an empty result set marked as successful. The coordinator produces a final report with no academic sources. What is the critical failure in this design?

B The subagent silently suppressed the access failure by returning empty results as success, preventing the coordinator from attempting recovery or noting the gap in coverage.

Silent suppression — returning empty results marked as success — is the worst error propagation anti-pattern. The coordinator believes the academic search succeeded and found nothing, so it produces a report without academic sources and without noting the coverage gap. The subagent should have returned structured error context with the failure type, what was attempted, and potential alternatives.

D The entire workflow should have been terminated when the academic database subagent encountered the 403 error.

Workflow termination on a single subagent failure wastes partial results from the web search and industry reports subagents that may have completed successfully. The coordinator should proceed with available results and annotate the coverage gap.

C The subagent should have retried the request with exponential backoff, as 403 errors are often transient.

403 Forbidden is a permission error, not a transient error. Retrying will not resolve it — the subagent lacks the required access credentials. The correct response is to propagate the error with its type so the coordinator knows the failure is not retryable.

A The coordinator should validate that all subagents return non-empty results before producing the final report.

Requiring non-empty results from all subagents would cause the workflow to fail when a subagent legitimately finds no matching data. The issue is that the error was suppressed, not that the coordinator failed to validate result volume.

WHERE THIS COMES FROM

Lesson 5.3: Error Propagation in Multi-Agent Systems (Silent suppression anti-pattern)
Lesson 5.3: Error Propagation in Multi-Agent Systems (Access failure vs empty)

Q46 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / CONTEXT-PASSING q-1-3-015 · Multi-Agent Research System **CORRECT**

A coordinator spawns a research subagent to investigate market trends. The subagent returns detailed findings. The coordinator then needs to pass these findings to a synthesis subagent for report writing. However, the synthesis subagent produces a report that contradicts the research findings. What is the most likely cause?

B The coordinator passes a summary instead of the full structured research output, so the synthesis agent fills gaps from its training data.

Subagents do not share memory. The coordinator must explicitly pass the complete research output to the synthesis agent. If the coordinator only passes a summary, the synthesis agent has no access to the detailed findings and will rely on its training data to fill gaps, potentially contradicting the actual research.

D The two subagents are using different model versions with different training data

Model version differences would not cause contradictions with explicitly provided research data. The issue is that the research data is not being fully passed to the synthesis agent, not that the agents have different base knowledge.

C The synthesis subagent has its own web search tool and keeps pulling contradictory information from the open web instead of the research it was handed.

A synthesis subagent should not have web search tools; that violates scoped tool access. The real issue is that the full research output is not being passed to it, not that it is searching the web.

A The synthesis subagent has a conflicting system prompt that overrides the research data

While possible, conflicting system prompts would produce consistently wrong outputs across all requests. The more common cause is that the coordinator is not passing the full research output, so the synthesis agent fills gaps with its own training data.

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Context passing)
Lesson 1.3: Subagent Invocation and Context Passing (Structured metadata format)

Q47 · D3 3.6 CI/CD-INTEGRATION / SESSION-ISOLATION q-3-6-005 · CI/CD Pipeline Integration **CORRECT**

A CI/CD pipeline runs three Claude Code steps in sequence: (1) generate a changelog from git commits, (2) review the changelog for accuracy, and (3) check for breaking changes. The team notices that step 2 never flags inaccuracies and step 3 misses obvious breaking changes that the changelog omits. What is the root cause?

B The CLAUDE.md file does not contain changelog formatting standards, so the review step has no criteria to evaluate against

Missing formatting standards would affect the quality of the generated changelog, not the review's ability to catch inaccuracies. The pattern of reviews consistently missing issues points to shared-context bias, not missing criteria.

C The -p flag is not being used, causing each step to wait for interactive input

If -p were missing, the pipeline would hang rather than produce output that misses issues. The steps are producing output (the changelog is generated, reviews complete), but the reviews are ineffective.

A The three steps share session context, so steps 2 and 3 inherit step 1's reasoning instead of judging the changelog

When review and analysis steps share context with the generation step, they inherit the original reasoning and are less likely to identify gaps or errors. Independent sessions force each step to evaluate the changelog from scratch without bias from the generation reasoning. Session context isolation is critical for CI/CD review pipelines.

D The steps need to use --output-format json so that each step can parse the previous step's structured output

Output format affects how results are structured, not whether reviews are thorough. JSON output would help with parsing but would not address the fundamental issue of review steps retaining generation context bias.

WHERE THIS COMES FROM

Lesson 3.6: CI/CD Integration (Session context isolation)

Q48 · D2 2.2 STRUCTURED-ERROR-RESPONSES / ERROR-CATEGORIES q-2-2-006 · Multi-Agent Research System **CORRECT**

A 'search_papers' MCP tool has three failure patterns: (1) the upstream academic API returns HTTP 503 during peak hours, (2) users request papers from a restricted journal the system has no licence for, and (3) the agent submits a malformed DOI like 'doi-1234' that fails the input regex. The team wants structured error metadata so agents can handle each case differently. Which 'errorCategory' and 'isRetryable' combination is correct for all three?

B HTTP 503: transient/retryable; restricted journal: business/not retryable; malformed DOI: validation/retryable.

HTTP 503 is a temporary upstream outage (transient, will likely resolve on retry). Restricted journal is a policy limitation (business, will never resolve on retry — escalate or offer an alternative source). Malformed DOI is validation; the agent must repair the input and then retry.

D HTTP 503: validation/retryable; restricted journal: business/not retryable; malformed DOI: business/not retryable.

HTTP 503 is transient (a temporary upstream outage), not validation — nothing is wrong with the request input. A malformed DOI is validation (fix input and retry), not business — business errors represent policy or rule violations, not input format problems.

A All three should be errorCategory: 'transient', isRetryable: true, since they all prevent the tool from completing its task.

Lumping every failure into 'transient' is the canonical anti-pattern. The restricted journal case is a business error (licence policy); the agent must escalate, not retry. The malformed DOI is validation; the agent must repair the input format first.

C HTTP 503: transient/retryable; restricted journal: transient/retryable; malformed DOI: transient/retryable.

The restricted journal error is not transient — the system lacks a licence, and no amount of retrying will grant access. A malformed DOI is validation, not transient; the agent must repair the input rather than blindly retry the same value.

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Four error categories)

Q49 · D1 1.6 TASK-DECOMPOSITION / COORDINATOR q-1-6-007 · Multi-Agent Research System **CORRECT**

A multi-agent research system must process a customer's request that involves three sequential stages: data collection, analysis, and report generation. Each stage depends on the output of the previous one. Which orchestration pattern is most appropriate?

D Hub-and-spoke with all three agents reporting independently to the coordinator

Independent reporting ignores the sequential dependencies. The analysis agent cannot produce meaningful results without data collection output. The pipeline must enforce the execution order.

C Dynamic adaptive decomposition — let the coordinator decide the order at runtime based on query complexity

Dynamic adaptive decomposition is for open-ended investigation tasks where the next step depends on discoveries. Here, the three stages and their dependencies are known in advance, making a fixed pipeline the more predictable and efficient choice.

B Pipeline orchestration — pass the output of each stage as input to the next in a defined sequence

Pipeline orchestration is the correct pattern for sequential dependencies. Each stage completes before the next begins, with the output of one stage serving as the input to the next. This matches the data collection → analysis → report generation dependency chain.

A Parallel orchestration — run all three subagents simultaneously to minimise latency

Parallel execution requires independent tasks. Here, analysis depends on data collection results and report generation depends on analysis results. Running them simultaneously would mean the analysis agent has no data to analyse.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Q50 · D3 3.3 PATH-SPECIFIC-RULES / MULTI-RULESET q-3-3-002 · CI/CD Pipeline Integration

INCORRECT

A DevOps team's polyglot codebase has Terraform files in terraform/, Kubernetes manifests in k8s/, and Dockerfiles scattered throughout various service directories. They want Claude Code to automatically apply infrastructure-specific conventions when editing each type of file, without loading all conventions for every session. What is the correct approach?

D Create three path-scoped .claude/rules/ files (terraform.md, kubernetes.md, docker.md), each targeting its own file types

Each rule file targets specific infrastructure file types with appropriate glob patterns. Terraform conventions load only when editing Terraform files, Kubernetes conventions only for k8s manifests, and Docker conventions only for Dockerfiles — regardless of which directory the Dockerfiles are in. This provides token-efficient conditional loading for all three file types. (correct answer)

B Add all infrastructure conventions to the root CLAUDE.md with clear section headings so the model knows which section applies

Root CLAUDE.md loads for every session. All infrastructure conventions would consume tokens even when editing TypeScript application code. Section headings do not prevent the conventions from being loaded — the model still processes the entire file.

A Create three directory-level CLAUDE.md files: one in terraform/, one in k8s/, and one in the project root for Docker conventions

Directory-level CLAUDE.md files in terraform/ and k8s/ would work for those specific directories, but Dockerfiles are scattered throughout various service directories. A single root CLAUDE.md for Docker conventions would load for every session, defeating the token efficiency goal. This approach does not handle the cross-directory Dockerfile pattern.

C Create a single .claude/rules/infrastructure.md file with paths: ["**/*"] containing all infrastructure conventions

The glob pattern **/* matches every file in the codebase, which means the infrastructure conventions would load for every session — exactly the same as putting them in root CLAUDE.md. This defeats the purpose of conditional loading. (your answer)

WHERE THIS COMES FROM

Lesson 3.3: Path-Specific Rules (Multiple rule files)

Lesson 3.3: Path-Specific Rules (Glob patterns)

Q51 · D2 2.3 TOOL-DISTRIBUTION-CHOICE / TOOL-CHOICE q-2-3-003

CORRECT

A developer productivity platform uses an automated documentation agent. The agent must always extract metadata from source files before generating documentation, but in testing it sometimes skips the metadata extraction step and produces documentation from incomplete information. What is the correct tool_choice configuration to enforce the mandatory first step?

C Set tool_choice to forced selection with the specific tool name extract_metadata for the first turn, then switch to 'auto' for subsequent turns.

Forced selection `{'type': 'tool', 'name': 'extract_metadata'}` ensures the model must call this specific tool. The model cannot skip it or choose a different tool. After the first turn completes, switching to 'auto' allows normal operation for documentation generation.

B Set `tool_choice` to 'any' so the model must call a tool, relying on the tool description to guide it to `extract_metadata` first.

'any' forces the model to call a tool but lets it choose which one. It may select a different tool first, still skipping `extract_metadata`. This guarantees a tool call but not the correct tool call.

A Set `tool_choice` to 'auto' and add a system prompt instruction: 'You must call `extract_metadata` before any other tool.'

With 'auto', the model decides whether to call a tool at all. A system prompt instruction is advisory, not enforced — the model can still skip the step. This does not guarantee the mandatory first step.

D Remove all other tools except `extract_metadata` from the agent's configuration so it has no choice.

This prevents the agent from performing any other task after metadata extraction. The agent needs access to documentation generation tools for subsequent steps. Forced tool selection achieves the constraint without crippling the agent.

WHERE THIS COMES FROM

Lesson 2.3: Tool Distribution and Tool Choice (`tool_choice` configuration)

Anthropic: Tool Use Documentation ↗

Q52 · D3 3.3 PATH-SPECIFIC-RULES / DIRECTORY-GLOBS q-3-3-010 · Technical Documentation Maintenance System

CORRECT

A docs team maintains three content types in `'docs/api/'`, `'docs/architecture/'`, and `'docs/runbooks/'`, each with distinct formatting, terminology, and required sections. They want Claude Code to apply the correct standards automatically when editing any docs file, without loading all three rule sets in every session. What is the correct configuration architecture?

B Create three rule files in `.claude/rules/` with YAML frontmatter paths targeting each directory: paths: `['docs/api/**']` for API rules, paths: `['docs/architecture/**']` for architecture rules, and paths: `['docs/runbooks/**']` for runbook rules

Path-scoped rules in `.claude/rules/` with glob patterns load only when editing files in the matching directory. Each documentation type gets its own rules loaded automatically and exclusively, conserving context tokens and eliminating the risk of applying incorrect standards.

D Create three custom skills (`/api-docs`, `/arch-docs`, `/runbook-docs`) and require writers to invoke the correct one before editing

This scenario explicitly requires writers to invoke the right skill manually, which depends on human memory and discipline — the wrong pick silently applies the wrong standards. Even if the skills added a paths frontmatter to auto-activate, they'd still load as a task-style workflow rather than as always-in-context guidance. Path-scoped rules in `.claude/rules/` are the correct choice: they load into context automatically when Claude reads a matching file, with no human step in the loop.

C Place a separate `CLAUDE.md` file in each of `docs/api/`, `docs/architecture/`, and `docs/runbooks/` with the type-specific rules

Directory-level `CLAUDE.md` files load based on the working directory hierarchy, but Claude Code sessions typically operate from the repository root. These directory-level files would only load when the agent is directly working within that specific directory, which is less reliable than path-scoped rules that match by file pattern regardless of working directory.

A Place all three sets of rules in the root `CLAUDE.md` with clear section headings so Claude Code can identify which rules apply

Root `CLAUDE.md` loads for every session. Including all three rule sets wastes context tokens when editing only one documentation type. The model must also correctly identify which section applies, adding an unnecessary point of failure.

WHERE THIS COMES FROM

Lesson 3.3: Path-Specific Rules (Practical rule examples)

Lesson 3.3: Path-Specific Rules (Path-specific rules)

Q53 · D2 2.5 BUILT-IN-TOOLS / GREP-VS-GLOB q-2-5-001

CORRECT

A developer needs to find all files that call a deprecated function `processLegacyOrder()` and also find all test files for those callers. Which tool sequence is correct?

A Glob for `*processLegacyOrder*` to find callers, then Grep for test files.

Glob matches file paths, not file contents. It cannot find function callers — it would only match files named after the function, which is unlikely.

C Grep for `'processLegacyOrder'` to find the callers, then Glob to match their test files by name.

Grep searches file contents, which is correct for finding function callers. Glob matches file paths, which is correct for finding test files by naming pattern such as `*/test.tsx`. This is the optimal two-step sequence.

B Read every source file to search for the function calls, then Read every test file to confirm the callers.

Reading all files upfront is a context-budget killer. It consumes tokens on irrelevant files and is the exact anti-pattern the exam penalises.

D Bash with `'find . -name "*.ts" | xargs grep processLegacyOrder'` for both steps.

While technically functional, this bypasses the built-in tools designed for these tasks. The exam expects candidates to use the right built-in tool for each task.

WHERE THIS COMES FROM

Lesson 2.5: Built-in Tools (Grep vs Glob)

Claude Code: Built-in Tools ↗

Q54 · D2 2.2 STRUCTURED-ERROR-RESPONSES / ACCESS-VS-EMPTY q-2-2-010

INCORRECT

A customer support agent calls a payment processing tool to check a refund status. The tool returns: `{"status": "success", "data": [], "message": "No refunds found for this transaction"}`. The agent's recovery logic treats this as an error and retries the request. What is wrong with the agent's behaviour?

C The tool should not return an empty array — it should return an error with `isRetryable: false` to prevent the retry.

The tool response is correct — a successful query with no results should return an empty array with a success status. The problem is the agent's recovery logic failing to distinguish valid empty results from actual errors. (your answer)

B The agent wrongly treats a valid empty result as a failure, not a successful query that found no records.

An empty array result is not a failure when the status indicates success. The tool queried the payment system, found no refund records, and returned that information correctly. The agent should inform the customer that no refund exists for this transaction, not retry. (correct answer)

A The agent should retry more times, as the payment system may be slow to process refund records.

The tool returned status 'success' with a clear message. This is not a slow-processing issue — the query completed successfully and found no refunds. Retrying will produce the same result.

D The agent should escalate to a human immediately instead of retrying, as payment queries should never be retried.

There is nothing to escalate. The query succeeded and returned a valid result: no refunds exist. The agent should simply communicate this to the customer.

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Access failure vs empty result)

Q55 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / STRUCTURED-METADATA q-1-3-001 · Multi-Agent Research System CORRECT

A synthesis agent produces a report with several claims that have no source attribution. The web search and document analysis subagents are working correctly and returning well-sourced results. What is the most likely root cause?

A The synthesis agent's system prompt does not instruct it to include citations, so it drops source attribution even when given well-sourced input.

Even with a citation instruction, the synthesis agent cannot attribute claims to sources that were never passed to it. The lesson is explicit: do not blame the synthesis prompt; the root cause is the coordinator dropping structured metadata.

C The web search subagent needs to return results in a different format

The web search subagent is working correctly. The issue is how the coordinator passes its results to the synthesis agent.

B The coordinator passes content to the synthesis agent stripped of structured metadata like source URLs and document names.

Context passing must include structured data that separates content from metadata. Without source URLs and document names, the synthesis agent has no attribution information to include.

D The synthesis agent needs access to the web search tool directly

Giving the synthesis agent web search tools violates the principle of scoped tool access and does not address the context passing failure.

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Structured metadata format)

Lesson 1.3: Subagent Invocation and Context Passing (Context passing)

Q56 · D5 5.1 CONTEXT-WINDOW-MANAGEMENT / POSITION-EFFECTS q-5-1-010 · Multi-Agent Research System **CORRECT**

A multi-agent system runs a 45-minute deep research workflow. Around the 30-minute mark, the coordinator's synthesis quality drops: it refers to 'the key findings' instead of specific statistics it cited earlier, and misattributes a claim from the financial subagent to the news subagent. No token limits or errors have been hit. What is the most likely diagnosis and correct mitigation?

B Context degradation: early results are buried deep in a long context. Consolidate key findings into a structured block near the end.

Context degradation occurs when important information is buried deep in a long conversation. The model's attention to early content diminishes as new content accumulates. Consolidating key findings into a recent, structured block keeps them in the model's effective attention window without losing the specifics.

A The model is experiencing 'fatigue' from a long session and needs a cooldown period before continuing

LLMs do not experience fatigue. Each inference is independent. The degradation is caused by context window dynamics, not model tiredness.

C The token limit has been silently exceeded and the API is truncating early messages. Upgrade to a larger context window model

The question states no token limits have been hit. Context degradation occurs well before token limits are reached — it is an attention quality issue, not a capacity issue.

D The subagents are returning inconsistent data, confusing the coordinator. Add data validation to each subagent's output before it reaches the coordinator

The coordinator previously cited these same findings correctly earlier in the session. The issue is not data quality from subagents but the coordinator's degrading attention to that data over time.

WHERE THIS COMES FROM

Lesson 5.4: Codebase Exploration and Context Degradation (Context degradation)

Lesson 5.4: Codebase Exploration and Context Degradation (Summary injection)

Q57 · D4 4.4 VALIDATION-RETRY / RETRY-BOUNDARY q-4-4-003 · Content Moderation and Classification System **CORRECT**

The moderation system validates that every classification includes a 'reasoning' field explaining the decision. When validation fails (empty reasoning), the system retries. For most posts, the retry succeeds after feeding back the error 'reasoning field was empty — provide a brief justification for the classification.' However, for posts in unfamiliar languages, retries consistently fail. What should the system do?

D Remove the reasoning validation requirement for posts in unfamiliar languages

Removing the validation requirement means these posts would receive classifications without any justification, which undermines auditability. The correct approach is human review, not lower standards.

- A** Increase the retry count from 1 to 5 for unfamiliar language posts since the model may succeed with more attempts

More retries are ineffective when the model lacks the capability to analyse content in that language. The failure is not a transient format issue — the model genuinely cannot provide reasoning for content it cannot understand.

- C** Add the unfamiliar language to the system prompt as a supported language to encourage the model to try harder

Listing an unsupported language in the prompt does not give the model language understanding. This may actually worsen the problem by encouraging fabricated analysis of content the model cannot read.

- B** Retry format errors on analysable posts; route capability gaps like unfamiliar languages to human review.

The retry boundary distinguishes between fixable format issues and unfixable capability gaps. Empty reasoning on analysable posts is a format error that retries with error feedback can fix. Unfamiliar languages represent absent capability — no amount of retrying will create language understanding. Route these to human review.

WHERE THIS COMES FROM

Lesson 4.4: Validation, Retry, and Feedback Loops (Retry boundary)

Q58 · D3 3.6 CI/CD-INTEGRATION / STRUCTURED-VALIDATION q-3-6-007 · Technical Documentation Maintenance System

CORRECT

A CI pipeline validates that generated API docs match the codebase. It runs `claude -p 'Verify that all public API endpoints in src/api/ have corresponding documentation in docs/api/'` with `--output-format json`. The check consistently reports 100% coverage, yet manual audits regularly find undocumented endpoints. What is the most likely cause?

- A** The `--output-format json` flag corrupts the analysis results, causing false positives

`--output-format json` controls the output structure, not the analysis logic. It formats results as JSON for downstream parsing but does not affect the accuracy of the verification itself.

- B** Claude Code is doing a shallow pattern match, not a deep semantic comparison, and the prompt lacks validation criteria

Without explicit validation criteria (e.g., 'for each exported function in `src/api/`, check that `docs/api/` contains a section with the function name, parameters, and return type'), the model may perform a shallow check. Adding structured criteria, expected output format, and examples of what constitutes a gap produces reliable verification.

- D** The CI pipeline needs a separate review step where a second Claude Code instance verifies the first instance's findings

Adding a second instance to review the first adds latency and cost without addressing the root cause. If the first instance's prompt lacks clear validation criteria, the second instance reviewing the same vague output will draw the same flawed conclusions.

- C** The `-p` flag prevents Claude Code from reading files, so it generates a plausible-sounding report without actual file access

The `-p` flag runs Claude Code in non-interactive (pipe) mode. It does not restrict file access — Claude Code retains full access to its built-in tools (Read, Grep, Glob) for reading files.

WHERE THIS COMES FROM

Lesson 3.6: CI/CD Integration (Structured output and validation)

Lesson 3.6: CI/CD Integration (CLAUDE.md for CI context)

Q59 · D2 2.2 STRUCTURED-ERROR-RESPONSES / ACCESS-VS-EMPTY q-2-2-001

CORRECT

A tool returns an empty array after a customer lookup. The agent retries 3 times, then escalates to a human agent. Analysis shows the customer's account simply does not exist. What is the root cause of this wasted effort?

- D** The system prompt should instruct the agent not to retry customer lookups.

Hard-coding retry rules per tool in the system prompt is brittle. The proper fix is structured error metadata that tells the agent whether the result is retryable.

A The retry limit is too low — increasing to 5 retries would resolve the issue.

More retries make the problem worse. The tool succeeded — it found no matching customer. Retrying a successful query with no matches will never produce different results.

C The escalation threshold is too aggressive — the agent should exhaust more retries before involving a human.

The problem is not the escalation threshold. The problem is that the agent retries at all. A valid empty result requires no retry and no escalation.

B The tool does not distinguish between access failures and valid empty results, so the agent treats 'no matches' as a retrievable failure.

The tool successfully queried the data source and found no matches. This is a valid empty result, not an access failure. The agent's recovery logic cannot distinguish the two, causing it to retry when it should accept the result.

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Access failure vs empty result)

Q60 · D2 2.3 TOOL-DISTRIBUTION-CHOICE / CONSOLIDATION q-2-3-008

CORRECT

The data platform team has built an MCP server with 22 tools: `query_snowflake`, `query_postgres`, `query_api`, plus 19 specialised tools for individual data transformations (`pivot_table`, `calculate_percentile`, `normalise_currency`, etc.). Agents take 3-4 turns to select the correct tool and frequently choose the wrong transformation. What is the most effective redesign?

B Consolidate the 19 transformation tools into a single `transform_data` tool with a `transform_type` parameter, reducing the total to 4 tools.

Reducing from 22 to 4 tools brings the count within the optimal 4-5 range for reliable tool selection. The 19 transformation tools share a common pattern (input data, transformation type, output format) and are natural candidates for consolidation into a single parameterised tool.

C Use `tool_choice: 'any'` to force the agent to always call a tool, eliminating turns where the agent reasons without acting.

Forcing tool calls does not improve selection accuracy — it just ensures the agent picks something, potentially the wrong tool. The root cause is too many similar tools, not too few tool calls.

A Improve all 22 tool descriptions with detailed examples and boundary conditions to help the agent distinguish between them.

Better descriptions help, but 22 tools still exceeds the practical limit for reliable selection. Research shows tool selection degrades significantly beyond 4-5 tools per agent. The tool count itself is the problem.

D Split the tools across two separate MCP servers — one for queries and one for transformations — to reduce cognitive load.

MCP server boundaries are invisible to the agent. All tools from all connected servers appear in a single list. Splitting across servers does not reduce the number of tools the agent must choose from.

WHERE THIS COMES FROM

Lesson 2.3: Tool Distribution and Tool Choice (Consolidating near-duplicate tools)

Lesson 2.3: Tool Distribution and Tool Choice (Tool overload)

SUPPORT

Found this useful?

The site is free and always will be. If the mock exam helped, you can buy me a coffee. Support goes to keeping the lights on.

☞ BUY ME A COFFEE